

تقرير الأخطاء الشامل

مختبر الأخطاء | H Wallet | المستخدم ولوحة الإدارة

تحليل هيكلي كامل للازدواجيات والأخطاء البرمجية
والفجوات في نظام API وإدارة الرصيد وتدفق الشراء

تاريخ التقرير: 16 يونيو 2026

المشروع: Wallet-for-working

عدد الأخطاء الحرجة: 14 | عدد الأخطاء المتوسطة: 11 | أخطاء بسيطة: 4

فهرس المحتويات

1 ملخص تنفيذي - الإحصائيات العامة

2 أخطاء تطبيق المستخدم - الازدواجية في الأقسام

3 أخطاء تطبيق المستخدم - نظام عرض الخدمات والأيقونات

4 أخطاء تطبيق المستخدم - نظام الإيداع والسحب والتحويل

5 أخطاء تطبيق الأدمن - الازدواجية في اللوحات

6 أخطاء تطبيق الأدمن - مسارات Firebase المتضاربة

7 نظام API - كيفية اتصال المزود وإدارة الرصيد

8 تدفق الشراء - ماذا يحدث عند شراء المستخدم

9 ماذا يحدث عند نفاذ رصيد API

10 فجوة التصميم مقابل التنفيذ - قاعدة البيانات

11 نظام العملات - مصمم لكن غير مفعّل

12 خطة الإصلاح الشاملة

1. ملخص تنفيذي

إحصائيات الأخطاء

3

أنظمة بيانات متوازية

7

ازدواجية لوحات

4

خطأ بسيط

11

خطأ متوسط

14

خطأ حرج

5

مميزات مفقودة

بعد تحليل شامل للمشروع بالكامل، تبين وجود مشاكل هيكلية عميقة تؤثر على جميع جوانب التطبيق. المشكلة الجذرية تكمن في أن المشروع يستخدم ثلاثة أنظمة بيانات متوازية (Firebase RTDB + Supabase PostgreSQL + Prisma SQLite) دون أي آلية مزامنة بينها. هذا أدى إلى ازدواجيات شديدة في واجهة المستخدم ولوحة الإدارة، وغياب كامل لنظام تدفق الشراء عبر API، وعدم وجود طريقة للأدمن لإدارة رصيده في مزود API.

الأخطاء تنقسم إلى أربع فئات رئيسية: (1) ازدواجية في تعريف الأقسام والخدمات عبر ملفات متعددة بتسميات مختلفة، (2) لوحات إدارية مكررة تؤدي نفس الوظيفة بسجلات بيانات مختلفة، (3) غياب كامل لنظام شراء تلقائي عبر API وآلية إيداع رصيد، (4) فجوة كبيرة بين تصميم قاعدة البيانات والتنفيذ الفعلي حيث أن جميع دوال RPC المعرّفة في Supabase لا تُستخدم أبداً.

2. أخطاء تطبيق المستخدم - الازدواجية في الأقسام

2.1 تعريف الأقسام في ثلاثة ملفات مختلفة

الأقسام والخدمات معرّفة بشكل مستقل في ثلاثة ملفات بتصنيفات وتسميات غير متسقة. هذا يعني أن إضافة قسم جديد في ملف واحد لا يظهره تلقائياً في الملفات الأخرى.

معرف القسم	home-screen.tsx	store.ts	category-detail-screen.tsx
service-providers	مزودين الخدمات	مزودين الخدمات	موجود (فارغ)

wallet-services	خدمات المحفظة	خدمات المحفظة الخاصة بنا	8 أقسام فرعية
telecom	الاتصالات	الاتصالات	قسمان فرعيان
government	غير موجود	خدمات حكومية	قسمان فرعيان
entertainment	غير موجود	غير موجود	5 أقسام فرعية
cards	غير موجود	غير موجود	3 أقسام فرعية
transfer	تنقل خاص	غير موجود	غير موجود
recharge	تنقل خاص	غير موجود	غير موجود

حرج أقسام محذوفة ما زالت موجودة: قسم الخدمات الحكومية ([government](#)) موجود في store.ts و category- detail-screen.tsx لكنه غير معرّف في شاشة الرئيسية. قسم الترفيه ([entertainment](#)) والبطاقات ([cards](#)) موجودان فقط كبيانات احتياطية في category-detail-screen.tsx.

[home-screen.tsx:90-102](#) | [store.ts:577-586](#) | [category-detail-screen.tsx:59-104](#)

2.2 أقسام فرعية مكررة بين الفئات

في ملف [category-detail-screen.tsx](#)، هناك تكرار حرفي للأقسام الفرعية بين الفئات المختلفة:

القسم الفرعي	wallet-services	entertainment	cards
shooting (ألعاب إطلاق النار)	موجود	مكرر	-
strategy (ألعاب الاستراتيجية)	موجود	مكرر	-
adventure (ألعاب المغامرات)	موجود	مكرر	-
platforms (منصات الألعاب)	موجود	مكرر	-
streaming (خدمات البث)	موجود	مكرر	-
store-cards (بطاقات المتاجر)	موجود	-	مكرر
gaming-cards (بطاقات الألعاب)	موجود	-	مكرر
payment-cards (بطاقات الدفع)	موجود	-	مكرر

حرج 5 أقسام فرعية مكررة بين wallet-services و entertainment، و 3 أقسام فرعية مكررة بين wallet-services و cards. هذا يعني أن المستخدم يرى نفس الخدمات في أقسام مختلفة.

2.3 قوائم شركات الاتصالات المكررة

شركات الاتصالات معروفة بشكل مستقل في ثلاثة ملفات مختلفة مع تناقضات:

الملف	الشركات	ملاحظة
recharge-screen.tsx:33-39	YM, Yo, Sabafon, Y, YNet	5 شركات
charging-companies-screen.tsx:14-21	YM, Yo, Sabafon, Y, YNet, YTG	6 شركات - توجد YTG الإضافية
services-screen.tsx:174	YM, Yo, Sabafon, Y	4 شركات فقط - ناقصة

تحذير شركة YTG (واي تي جي) موجودة في `charging-companies-screen` فقط ومفقودة من الشاشتين الآخرين.

2.4 شاشات غير متصلة بالتنقل

حرج شاشة الأدمن غير متصلة: `setActiveScreen('admin')` يُستدعى من `home-screen.tsx` (نقر 5 مرات مخفي) ومن `admin-notifications.tsx`, لكن `overlayScreens` في `page.tsx` لا تحتوي على مفتاح `'admin'`. النتيجة: التنقل لشاشة الأدمن لا يعمل.
[page.tsx:651-674](#) | [home-screen.tsx:304](#)

تحذير شاشة `charging-companies` يتيمية: مسجلة في `overlayScreens` لكن لا يوجد أي مسار تنقل إليها. شاشة ميتة لا يمكن للمستخدم الوصول إليها.
[page.tsx:664](#) | [charging-companies-screen.tsx](#)

3. نظام عرض الخدمات والأيقونات

3.1 المنتجات مُرمّزة بشكل ثابت وليست من قاعدة البيانات

ملف `src/lib/products-data.ts` (أكثر من 1000 سطر) يحتوي على جميع المنتجات مكتوبة يدوياً بأسعار ثابتة بالريال اليمني بسعر صرف جامد 1 دولار = 1550 ريال مع هامش 3%. هذا الملف يُدمج مع بيانات Firebase في `order-bottom-sheet.tsx`.

حرج تعارض البيانات: المنتجات تأتي من مصدرين: Zustand store (Firebase) وملف `products-data.ts` المرمّز يدوياً. عند الدمج، قد يحل المنتج المرمّز يدوياً محل المنتج من قاعدة البيانات، مما يسبب أسعاراً خاطئة أو منتجات

3.2 خدمات الشاشة تقرأ من Firebase وليس Supabase

شاشة `services-screen.tsx` تُنشئ 5 مستمعي Firebase RTDB منفصلين لـ (`sections`, `providers`, `walletServices`, `sections`, `sub_sections`, `service_providers`)، رغم أن Supabase يحتوي على جداول (`sections`, `sub_sections`, `service_providers`)، فإن الدوال `()supabaseService.getServiceProviders` و `()supabaseService.getSections` و `()supabaseService.getProductPackages` لا تُستدعى أبداً من أي شاشة موجهة للمستخدم.

3.3 نظام أيقونات مزدوج ومتضارب

العدد	الغرض	الملف
SVG 15~	خدمات رئيسية + أيقونات فئات	<code>src/lib/service-icons.ts</code>
SVG +40~	اتصالات + ألعاب + بطاقات + متنوع	<code>src/lib/product-icons.ts</code>

بعض المفاتيح موجودة في كلا الملفين برموز SVG مختلفة، مما يسبب تضارب:

```
;const fallbackIconSrc = productIcons[service.iconKey] || serviceIcons[service.iconKey]
```

تحذير صور خارجية غير موثوقة: `category-detail-screen.tsx` يحتوي على روابط CDN خارجية من `codashop`, `seagmcdn`, `payermx`, `eneba`. هذه الروابط قد تتغير أو تنقطع بدون آلية بديلة.

`category-detail-screen.tsx:107-135`

3.4 خدمات المزودين لا تظهر بشكل صحيح

السبب الرئيسي لعدم ظهور خدمات المزودين هو أن البيانات تأتي من Firebase فقط. عند مزامنة مزود API عبر `()SyncProviderToFirebase` في `api-provider.ts`، يتم إنشاء مسار معقد:

1 مزود API يُضاف في لوحة الإدارة



2 مزامنة إلى `/Firebase: adminSettings/apiProviders/{id}`



3 إنشاء `/providers` في `/Firebase + packages/ + sections`

4 شاشة الخدمات تقرأ من providers / و sections / - لكن المسارات غير متطابقة

5 الأيقونات: لا توجد آلية لسحب أيقونات المنتجات من API - تستخدم أيقونات افتراضية فقط

4. نظام الإيداع والسحب والتحويل

4.1 الإيداع والسحب يحفظان محلياً فقط

حرج طلبات الإيداع والسحب تُحفظ في Zustand store فقط (حالة محلية) ولا تُكتب أبداً إلى Supabase. الدوال `supabaseService.createDepositRequest()` و `supabaseService.createWithdrawRequest()` موجودة لكنها لا تُستدعى أبداً. هذا يعني أن الطلبات تُفقد إذا مسح المستخدم بيانات المتصفح، وأن لوحة الإدارة التي تعتمد على Supabase لا تستطيع رؤية هذه الطلبات.

[deposit-screen.tsx:562,636](#)

4.2 عدم تناسق في حفظ البيانات المالية

العملية	Firebase	Supabase	محلي فقط
الصرافة	نعم (رصيد)	نعم (RPC + معاملة)	نعم (كاش)
الإيداع	لا	لا	نعم فقط
السحب	لا	لا	نعم فقط
الشحن/الطلب	نعم (طلب + رصيد)	لا	نعم (كاش)
التحويل	نعم (رصيد + معاملة)	لا	نعم (كاش)

حرج شاشة الصرافة فقط تستخدم Supabase لتحديث الرصيد وإنشاء المعاملات. جميع العمليات المالية الأخرى تتجاوز Supabase تماماً.

4.3 حالة سباق في تحديث الرصيد أثناء الشحن

حرج في `recharge-screen.tsx:192-226`، الرصيد يُحسب من الحالة المحلية ويُكتب مباشرة إلى Firebase بدون عملية ذرية. إذا حدثت عمليتان في نفس الوقت، ستقوم العملية الثانية بالكتابة فوق تحديث العملية الأولى. شاشة الصرافة تستخدم `supabaseService.updateBalance()` (دالة RPC ذرية) بشكل صحيح، لكن الشحن لا يفعل ذلك.

5. أخطاء تطبيق الأدمن - الازدواجية في اللوحات

5.1 لوحة مزودي API غير متاحة (معزولة)

حرج لوحة `api-providers-panel.tsx` (1475 سطر - أكبر لوحة) غير متاحة من الشريط الجانبي! هذه اللوحة هي الأكثر تطوراً وتحتوي على: إضافة G2Bulk سريعة، فحص الرصيد، مخطط تاريخ الرصيد، تحذير رصيد منخفض، مزامنة مع Firebase و Supabase، وتصدير سجل الرصيد. لكنها غير مدرجة في الشريط الجانبي.

`sidebar.tsx:110-119 | api-providers-panel.tsx`

الحل: إضافة سطر واحد في `sidebar.tsx`

```
{ API', icon: Server, roles: ['owner'] 'مزودي':id: 'api-providers', label }
```

5.2 لوحات مكررة تتضارب

اللوحة الأولى	اللوحة الثانية	التضارب	التوصية
providers-panel	api-providers-panel	كلاهما يدير المزودين لكن بنماذج بيانات مختلفة. عند مزامنة API، يُنشئ مزودين في <code>/providers</code> - نفس مسار اللوحة الأولى.	دمج أو حذف providers-panel
notifications-panel	push-notifications-panel	كلاهما يرسل إشعارات. الثانية أقوى (FCM + شرائح + أنواع). كلاهما يكتب لنفس مسار <code>/adminNotifications</code> بتنسيقات مختلفة.	حذف notifications-panel
api-settings-panel	api-providers-panel	كلاهما ينشئ/يحرر مزودي API. لكن بتنسيقات بيانات مختلفة ستستبدل بيانات بعضها البعض في Firebase.	دمج في لوحة واحدة

حذف - مغطى بواسطة اللوحة الأكبر	يقرأ من مسار خاطئ <code>/apiProviders</code> بدلاً من <code>/adminSettings/apiProviders</code> - سيعرض دائماً فارغاً.	المزامنة داخل api-providers-panel	api-sync-panel api-sync-panel.tsx
دمج في لوحة واحدة بأقسام	الأولى لرسم المعاملات، الثانية لعمولات المزودين. تسمية مشوشة ومتداخلة.	commission-config-panel commission-config-panel.tsx	commissions-panel commissions-panel.tsx
إعادة تسمية للتوضيح	ليست مكررة فعلاً - الأولى للأدمن ينشئ أكواد، والثانية للمستخدمين. لكن الأسماء متشابهة.	user-gift-codes-panel user-gift-codes-panel.tsx	gift-codes-panel gift-codes-panel.tsx

5.3 مسارات Firebase المتضاربة

ثلاث لوحات تقرأ/تكتب مزودي API من مسارات مختلفة: [حرج](#)

اللوحة	مسار Firebase	قراءة/كتابة
api-providers-panel.tsx	<code>/adminSettings/apiProviders</code>	قراءة + كتابة
api-settings-panel.tsx	<code>/adminSettings/apiProviders</code>	قراءة + كتابة (تنسيق مختلف!)
api-sync-panel.tsx	<code>/apiProviders</code> (مسار مختلف!)	قراءة فقط

النتيجة: api-providers-panel و api-settings-panel يكتبان لنفس المسار بتنسيقات بيانات مختلفة فتستبدل بيانات بعضها البعض. و api-sync-panel يقرأ من مسار مختلف تماماً فسيظهر فارغاً دائماً.

سجل الرصيد في مسارين مختلفين: [حرج](#)

api-providers-panel يكتب إلى `/adminSettings/balanceLogs`
balance-log-panel يقرأ من `/apiBalanceLog`
النتيجة: لوحة سجل الرصيد ستظهر فارغة دائماً لأنها تقرأ من مسار مختلف عن الذي تُكتب إليه البيانات.

6. نظام API - كيفية اتصال المزود وإدارة الرصيد

6.1 البنية الحالية لاتصال API

ملف `src/lib/api-provider.ts` يحتوي على تنفيذ كامل لاتصال API:

الدالة	الغرض	تعمل؟
<code>()fetchProviderBalance</code>	جلب رصيد المزود عبر <code>getMe{baseUrl}</code>	نعم - لكن من الأدمن فقط
<code>()fetchProviderCategories</code>	جلب فئات المزود عبر <code>category{baseUrl}</code>	نعم - لكن من الأدمن فقط
<code>()fetchProviderProducts</code>	جلب منتجات المزود عبر <code>products?{baseUrl}</code> <code>category_id=X</code>	نعم - لكن من الأدمن فقط
<code>()purchaseProviderProduct</code>	شراء منتج عبر <code>products/{id}/purchase{baseUrl}</code>	موجودة لكن لا تُستدعى!
<code>()executeApiOrder</code>	تنفيذ طلب API عام مع قوالب	موجودة لكن لا تُستدعى بشكل صحيح!
<code>()syncProviderToSupabase</code>	مزامنة بيانات API إلى Supabase	موجودة لكن لا تُستدعى من العميل أبداً!
<code>()syncProviderToFirebase</code>	مزامنة بيانات API إلى Firebase	نعم - تُستدعى من الأدمن
<code>()checkProviderOrderStatus</code>	فحص حالة الطلب من API	موجودة لكن لا تُستدعى أبداً!

6.2 كيف يجب أن يعمل الاتصال



عند الشراء: فحص رصيد API أولاً - غير مفعّل!



7 تنفيذ الطلب عبر API - يعمل جزئياً بدون فحص رصيد



8 حساب العمولة وتسجيلها - غير مفعّل بالكامل!

6.3 المشكلة: لا توجد طريقة لإيداع رصيد في API

حرج لا توجد أي آلية للأدمن لإيداع أموال في حساب مزود API. الأدمن يستطيع فقط:

- عرض رصيد API (عبر `fetchProviderBalance`)
- عرض تاريخ الرصيد

لكنه لا يستطيع:

- إيداع/إضافة أموال إلى حساب API
 - إعداد شحن تلقائي عند انخفاض الرصيد
 - تحويل رصيد من محفظة التطبيق إلى حساب API
- عملياً، الأدمن يجب أن يذهب لموقع مزود API (مثل G2Bulk) ويضيف رصيداً يدوياً عبر طرق الدفع المتاحة لديهم. لا يوجد تكامل مالي مباشر بين التطبيق ومزود API.

الحل الموصى به

إضافة آلية في لوحة الإدارة تتضمن:

1. زر "شحن الرصيد" يفتح رابط موقع مزود API للدفع (حل بسيط)
2. تسجيل يدوي للإيداع: حقل يسمح للأدمن بتسجيل أنه أودع مبلغاً مع تاريخ وإيصال
3. فحص دوري للرصيد مع إشعار FCM عند الانخفاض عن حد معين
4. إيقاف تلقائي للخدمات عندما ينخفض رصيد API عن الحد الأدنى

7. تدفق الشراء - ماذا يحدث عند شراء المستخدم

7.1 التدفق الحالي (في order-bottom-sheet.tsx)



7.2 المشاكل في التدفق الحالي

حرج لا يوجد فحص لرصيد API قبل الشراء: المستخدم قد يدفع ثم يفشل الطلب لأن رصيد API غير كافٍ. رغم أن رصيده سيُعاد، إلا أن هذه تجربة سيئة.

حرج لا يوجد حساب للعمولة: المخطط يحتوي على commission_config و commission_log و product_packages.commission_amount لكن لا شيء من هذا يُحسب أثناء الشراء. العمولة الموجودة في الحزمة

هي مجرد "هامش ربح" (سعر البيع - سعر التكلفة) لكنها لا تُسجل أو تُحسب كعمولة فعلية.

حرج شاشة الشحن بدون أي اتصال **API: recharge-screen.tsx** تنشئ طلبات شحن دائماً بـ `executionType: manual` (يدوي). لا يوجد أي تكامل مع G2Bulk أو أي مزود خارجي لشحن الاتصالات.
[recharge-screen.tsx:210](#)

تحذير قراءة **Firestore مكررة**: عند تنفيذ طلب API، يقرأ إعداد المزود مباشرة من Firestore رغم أن البيانات متاحة بالفعل في Zustand store.
[order-bottom-sheet.tsx:127-192](#)

7.3 التدفق الصحيح المطلوب



8 إنشاء معاملة في Supabase transactions

9 متابعة حالة الطلب عبر checkProviderOrderStatus()

8. ماذا يحدث عند نفاذ رصيد API

لا توجد أي آلية للتعامل مع نفاذ رصيد API. حالياً عندما ينفد الرصيد:

السيناريو	ما يحدث حالياً	ما يجب أن يحدث
مستخدم يشتري منتج API	API يرجع خطأ → رصيد المستخدم يُعاد → لا إشعار للأدمن	فحص مسبق للرصيد + إشعار أدمن + إيقاف الخدمة تلقائياً
رصيد API أقل من الحد	تحذير مرئي فقط في لوحة الإدارة (لا يراه أحد لأن اللوحة معزولة!)	إشعار FCM فوري للأدمن + بريد إلكتروني + تعطيل تلقائي للخدمات
رصيد API = صفر	جميع الطلبات تفشل → المستخدمون يخسرون ثقة → لا يوجد بديل	تحويل تلقائي لمزود API بديل + إيقاف المنتجات المرتبطة
طلب ناجح جزئياً	لا يوجد تتبع لحالة الطلب بعد الإرسال	فحص دوري عبر checkProviderOrderStatus() + تحديث الحالة

لوحة api-providers-panel.tsx تحتوي على ثابت `LOW_BALANCE_THRESHOLD = 50` (سطر 79) لكنه يُستخدم للعرض فقط - لا يوجد إشعار أو إجراء تلقائي عند انخفاض الرصيد عن هذا الحد.

9. فجوة التصميم مقابل التنفيذ - قاعدة البيانات

9.1 ثلاثة أنظمة بيانات متوازية

المشروع يستخدم ثلاثة أنظمة بيانات في وقت واحد:

النظام	المستخدم	الجدول/المسارات	الحالة
--------	----------	-----------------	--------

مصمم بالكامل لكن معظمه لا يُستخدم	55 جدول	supabase.ts, لوحة الإدارة	Supabase (PostgreSQL)
المستخدم فعلياً في التشغيل	مسارات مثل /adminSettings/, /providers/, orders	firebase.ts, تدفق الطلبات, الشاشات	Firestore RTDB
قديم وغير مستخدم	13 نموذج	prisma/schema.prisma	Prisma (SQLite)

حرج التطبيق يعمل بشكل أساسي على Firestore RTDB وليس Supabase. مخطط Supabase هو وثيقة تصميم شاملة متصلة جزئياً فقط بالكود الفعلي. مخطط Prisma يبدو أنه تكرر أقدم.

9.2 دوال RPC محددة لكن لا تُستخدم أبداً

الدالة	معروفة؟	مستخدمة؟	المشكلة
<code>()update_user_balance</code>	نعم	لا أبداً	تحديثات الرصيد تتم عبر <code>runTransaction</code> في Firestore
<code>()create_transaction</code>	نعم	لا أبداً	المعاملات تُكتب مباشرة في Firestore
<code>()get_dashboard_stats</code>	نعم	ربما من الأدمن	تُرجع بيانات من <code>commission_log</code> الفارغ
<code>()convert_currency</code>	نعم	لا من التطبيق	أسعار الصرف تُقرأ من Firestore

9.3 جداول Supabase لا تُملأ أبداً

الجدول	الحالة	السبب
api_categories	فارغ	<code>syncProviderToSupabase</code> لا يُستدعى أبداً
api_products	فارغ	<code>syncProviderToSupabase</code> لا يُستدعى أبداً
api_balance_log	فارغ	السجلات تذهب لـ <code>adminSettings/balanceLogs</code> في Firestore
commission_log	فارغ	لا يوجد محرك عمولات
commission_config	فارغ	الإعدادات في <code>adminSettings/commissionConfig</code> في Firestore
orders	فارغ	الطلبات في <code>orders</code> في Firestore
deposit_requests	فارغ	الطلبات في <code>Zustand store</code> محلي فقط
withdraw_requests	فارغ	الطلبات في <code>Zustand store</code> محلي فقط

10. نظام العمولات - مصمم لكن غير مفعل

المشروع يحتوي على نظام عمولات مصمم بالكامل في قاعدة البيانات لكنه لا يعمل إطلاقاً في الكود:

المكون	موجود في المخطط	مفعل في الكود
commission_config (إعدادات العمولة)	نعم - 3 مستويات (عام/مزود/منتج)	لا - الإعدادات في Firebase فقط
commission_log (سجل العمولات)	نعم	لا - الجدول فارغ
product_packages.commission_amount	نعم	لا - يُعرض فقط كـ "هامش ربح"
product_packages.commission_type	نعم	لا
api_providers.default_commission	نعم	لا
حساب العمولة أثناء الشراء	مصمم	لا يحدث أبداً

حرج نظام العمولات غير موجود عملياً؛ لوحة commission-config-panel تكتب إعدادات إلى Firebase، لكن لا شيء يقرأ هذه الإعدادات أثناء الشراء. الأدمن لا يستطيع معرفة كم ربح من العمولات لأن commission_log فارغ دائماً.

11. خطة الإصلاح الشاملة

11.1 إصلاحات فورية (عاجلة)

#	الإصلاح	الملف	الصعوبة
1	إضافة api-providers للشريط الجانبي	sidebar.tsx	سهل - سطر واحد
2	إصلاح مسار balance-log-panel	balance-log-panel.tsx	سهل - تغيير مسار
3	إصلاح مسار api-sync-panel	api-sync-panel.tsx	سهل - تغيير مسار
4	إصلاح توجيه شاشة الأدمن	page.tsx	سهل - إضافة مفتاح

5	حذف notifications-panel (مكررة)	sidebar.tsx + page.tsx	سهل
6	حذف api-sync-panel (مسار خاطئ + مكررة)	sidebar.tsx + page.tsx	سهل

11.2 إصلاحات هيكلية (متوسطة)

#	الإصلاح	الوصف	الصعوبة
7	توحيد تعريف الأقسام	ملف واحد كمصدر وحيد للحقائق بدلاً من 3 ملفات	متوسط
8	دمج providers-panel و api-providers-panel	لوحة واحدة لإدارة جميع المزودين	متوسط
9	دمج api-providers-panel و api-settings-panel	لوحة واحدة شاملة لإعداد API	متوسط
10	دمج commissions-panel و commission-config-panel	لوحة واحدة بأقسام (رسوم + عمولات)	متوسط
11	إضافة إيداع/سحب إلى Supabase	استدعاء الدوال الموجودة من deposit-screen.tsx	متوسط
12	إزالة products-data.ts المرمّز يدوياً	الاعتماد على بيانات Firebase/Supabase فقط	متوسط

11.3 إصلاحات معمارية (كبيرة)

#	الإصلاح	الوصف	الصعوبة
13	بناء تدفق الشراء الكامل	فحص رصيد API + تنفيذ + حساب عمولة + تتبع حالة	كبير
14	إضافة آلية إدارة رصيد API	شحن يدوي + تسجيل + إشعارات + إيقاف تلقائي	كبير
15	تفعيل نظام العمولات	محرك حساب + تسجيل في commission_log + تقارير	كبير
16	الانتقال من Firebase إلى Supabase	استخدام RPCs الذرية بدلاً من runTransaction	كبير جداً
17	تفعيل نظام الترجمة	استبدال النصوص المرمّزة بـ i18n	كبير

11.4 ملخص أولويات الإصلاح

الخطوات الأربع الأهم

1. توحيد مصدر البيانات: اختيار Supabase كمصدر أساسي وإيقاف Firebase تدريجياً. هذا يحل مشاكل الازدواجية والتضارب.

2. بناء تدفق الشراء: إضافة فحص رصيد API قبل الشراء + حساب العمولات + تتبع حالة الطلب.
 3. إدارة رصيد API: آلية للأدمن لتتبع رصيده في API + إشعارات عند الانخفاض + إيقاف تلقائي.
 4. إزالة الازدواجيات: حذف اللوحات المكررة وتوحيد تعريفات الأقسام في ملف واحد.
-